BILDUNGSCAMPUS HEILBRONN · LIVE LORAWAN DATA

BeeHappy — Data Foundation

~4 hours | 7 exercises | pandas + matplotlib + Jupyter

Press Space to start →

1 · Intro

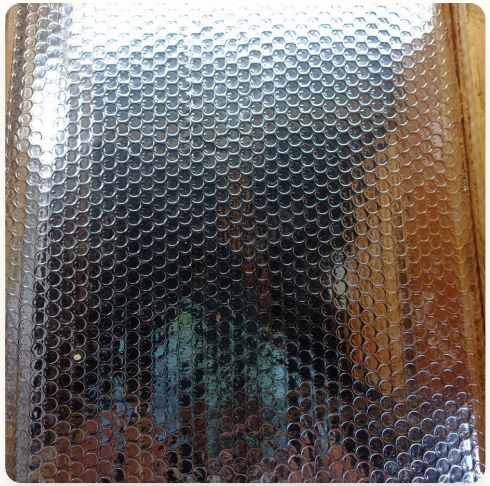
What the data is, where it comes from, and why cleaning it is the job

What is BeeHappy?

Three beehives at Bildungscampus Heilbronn, wired with **LoRaWAN sensors**, reporting continuously since **2025**. Each hive is wrapped in a different material.



Hive 1 · plastic film



Hive 2 · thermofolie



Hive 3 · cotton

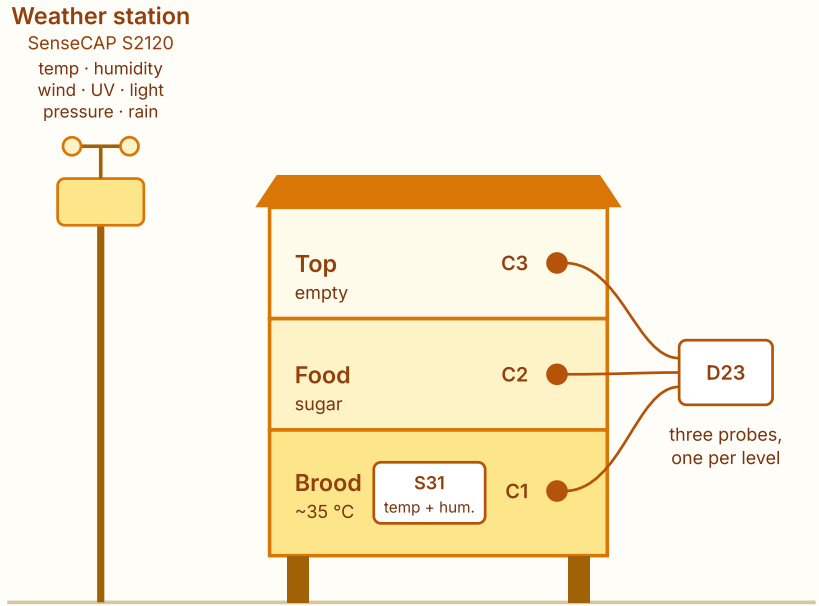
📡 What sensors do the hives carry?

Each hive carries devices that measure what is happening inside and around it:

- **Three probes** — C1 brood, C2 food, C3 empty top
- **Brood sensor** — temperature + humidity
- **Weather station** — temperature, humidity, wind, UV, pressure, light, rain



A LoRaWAN sensor mounted on a hive



Why and how we are collecting this data

TODAY

🐝 Which material is best for the colony?

- Whose brood stays closest to **35 °C**?
- Whose brood stays steadiest?

LATER · MACHINE LEARNING

🌡️ Colony health

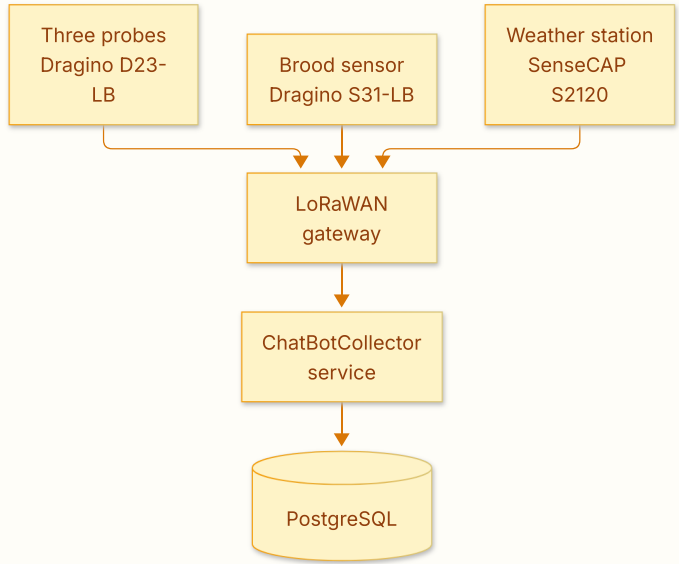
Alert when brood temperature drifts.

❄️ Overwintering

Flag hives at risk before winter losses.

Both need the same input: **one clean row per hive per hour**. That's what we build today.

HOW



The collector is a service with bugs.

Why data cleaning comes first

> A model can’t tell a signal from a mistake. It will happily learn the mistake.

Duplicates

Rows inserted twice silently skew every average.

Silent gaps

A missing row isn't a `NaN`. It's nothing, and nothing is hard to notice.

Typically 60–80% of a data science project is this step.

The data · two small lookup tables

BEEHIVES · 3 ROWS · THE HIVES

	beehive_id	name
0	1	Beehive No.1
1	2	Beehive No.2
2	3	Beehive No.3

+ `notes`, `location`, `installation_date`, `last_inspection_date`

SENSORS · 9 ROWS · THE DEVICES, AND WHICH HIVE EACH BELONGS TO

	sensor_id	beehive_id	sensor_name	sensor_type	measurement_units
0	1	1	LoRa-2CF7F1C0613005BC	LoRaWAN SenseCAP-S2120	lightIntensity,rainGauge,...
1	2	2	LoRa-2CF7F1C0613005BC	LoRaWAN SenseCAP-S2120	lightIntensity,rainGauge,...
2	3	3	LoRa-2CF7F1C0613005BC	LoRaWAN SenseCAP-S2120	lightIntensity,rainGauge,...
3	4	1	LoRa-A840411F645AE815	LoRaWAN Dragino-S31-LB	temperature,relativeHumidity
4	5	3	LoRa-A8404138A188669C	LoRaWAN Dragino-S31-LB	temperature,relativeHumidity
5	6	2	LoRa-A84041CC625AE81E	LoRaWAN Dragino-S31-LB	temperature,relativeHumidity
6	7	3	LoRa-A8404160C85A7A7B	LoRaWAN Dragino-D23-LB	tempC3,tempC1,tempC2
7	8	1	LoRa-A84041892E5A7A68	LoRaWAN Dragino-D23-LB	tempC3,tempC1,tempC2
8	9	2	LoRa-A840419521864618	LoRaWAN Dragino-D23-LB	tempC3,tempC1,tempC2

+ `installation_date`, `last_seen`, `is_active` · one weather station, one brood sensor and one three-probe device per hive

The data · one big table of readings

DATA · ~500K ROWS FOR THREE MONTHS · MILLIONS IN TOTAL

	reading_id	sensor_id	beehive_id	measurement_unit	ts	value
0	2293351	8	1	tempC3	2026-08-07 13:11:37.873	32.6
1	2293352	8	1	tempC1	2026-08-07 13:11:37.873	34.6
2	2293353	8	1	tempC2	2026-08-07 13:11:37.873	31.0
3	2293390	8	1	tempC3	2026-08-07 13:11:37.873	32.6
4	2293391	8	1	tempC1	2026-08-07 13:11:37.873	34.6
5	2293392	8	1	tempC2	2026-08-07 13:11:37.873	31.0
6	2293385	6	2	temperature	2026-08-07 13:19:12.026	33.8
7	2293386	6	2	relativeHumidity	2026-08-07 13:19:12.026	65.4

Long format: one row per *measurement*. A single uplink from probe device 8 writes three rows: `tempC1`, `tempC2`, `tempC3`.

HOW THE TABLES CONNECT

- data — sensor_id → sensors — which device, what type
- data — beehive_id → beehives — which hive
- sensors — beehive_id → beehives

You pull **the last three months** of `data`, filtered in SQL rather than in pandas. The full history is too big to be worth moving.

The target

One row per hive per hour.

Column	Meaning
`beehive_id`	which hive
`hour`	the hour, **UTC**
`brood_probe_temp`, `food_probe_temp`, `top_probe_temp`	three probes, one per level: brood (C1), food (C2), empty top (C3)
`brood_temp`, `brood_humidity`	temperature and humidity sensor in the brood level
`hour_local`, `month`, `dayofweek`	calendar, **Europe/Berlin**

Column	Meaning
`outside_temp`, `outside_humidity`	ambient
`outside_wind_speed`, `outside_wind_dir`	ambient
`outside_uv_index`, `outside_pressure_pa`	ambient
`outside_light`, `outside_rain`	ambient

The exercises

#	Exercise	What you produce	Time
0	**Extract**	Three tables, saved to disk	10 min
1	**Profile**	What is in the data	30 min
2	**Deduplicate**	One row per real reading	15 min
3	**Reshape & join**	A hive frame and a weather frame	60 min
4	**Align time & gaps**	One row per hive-hour	30 min
5	**Validate & document**	A validated file	10 min
6	**Look at what you built**	Plots that answer the question	30 min

2 · Your toolbox

Jupyter in five minutes, then pandas and matplotlib

Jupyter Notebook in five minutes

- A notebook is a list of **cells**: Markdown or Code.
- Run a cell → output appears below it.
- Variables **persist** between cells (the *kernel*).

> The last line is shown automatically: `df.head()` → a table.

Key	Does
`Shift` + `Enter`	run cell, go to next
`Ctrl` + `Enter`	run cell, stay
`Esc` then `A` / `B`	new cell above / below
`Esc` then `D` `D`	delete cell
`Esc` then `M` / `Y`	to Markdown / to Code
`Shift` + `Tab`	show the docstring

Jupyter: the one trap

In [1]:
```
data = pd.read_parquet("data.parquet")
data["temp"].mean()
```

```
347.0
```

FIX THE SCALE, THEN RUN THE SAME CELL AGAIN

In [3]:
```
data["temp"] = data["temp"] / 10
data["temp"].mean()
```

```
3.47  ✗ no error, no warning
```

Cells remember. Re-running a cell that changes data changes it again.

Fix: **Restart Kernel and Run All** before you trust a result.

3 · pandas 101

The functions you need

Why pandas?

Built for exactly this job

Missing values, duplicates, time zones, joins, reshaping.

The lingua franca

scikit-learn, matplotlib, seaborn, plotly all take a `DataFrame` directly.

Reads everything

`read_sql`, `read_parquet`, `read_csv`, `read_excel` — and a `to_*` for each.

Fifteen years of answers

Whatever you get stuck on, someone already asked it.

> Three months of readings is about half a million rows — it fits in memory, so the simplest tool is also the right one.

What pandas is

A DATAFRAME

In [1]:
```
import pandas as pd

df = pd.DataFrame({
    "sensor_id": [8, 8, 6],
    "unit": ["tempC1", "tempC2", "temperature"],
    "value": [35.5, 35.7, 35.6],
})
df
```

	sensor_id	unit	value
0	8	tempC1	35.5
1	8	tempC2	35.7
2	6	temperature	35.6

ONE COLUMN: A SERIES

In [2]:
```
df["value"]
```

```
0    35.5
1    35.7
2    35.6
Name: value, dtype: float64
```

In [3]:
```
type(df["value"])
```

```
pandas.Series
```

Row-oriented · transactional

Postgres stores each reading as a whole row. Fast to **write** one reading at a time.

Column-oriented · analytical

pandas and Parquet store each column together. Fast to **analyse**: `df["value"].mean()` reads one column.

Get, set, go.

1 · Install uv if you have not yet

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh
# Windows
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

2 · Create `.env` in the repo root, then replace `password` with the one we sent on Slack

```
cp .env.example .env        # macOS / Linux
copy .env.example .env      # Windows
```

3 · Open the notebook from the repo root

```
uv run jupyter notebook workshop/notebooks/workshop.ipynb
```

4 · Run the first cell. It prints your database URL.

.ENV

```
DATABASE_URL=postgresql://postgres:password@crossove
r.proxy.rlwy.net:45838/railway
```

Two kinds of cells in the notebook

```
# EXECUTE: working code, just run it
sensors
```

```
# TODO: your turn, replace the ...
raw_sensor_readings = ...
```

Exercise 0 · Extract

Pull once. Save. Work from your copy.

`pd.DataFrame` · `to_parquet` · `read_parquet`

Connecting to the database

```
import os, psycopg
from dotenv import load_dotenv

load_dotenv(REPO / ".env")
DATABASE_URL = os.environ["DATABASE_URL"]

with psycopg.connect(DATABASE_URL,
                     connect_timeout=30) as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT * FROM beehives")
        rows = cur.fetchall()
        cols = [c.name for c in cur.description]
```

`load_dotenv` reads `.env` into the environment.

`psycopg.connect` opens the connection. `with` closes it for you, even if a cell raises.

`cur.execute` sends the SQL, `cur.fetchall` pulls the rows back as a list of tuples, and `cur.description` carries the column names.

```
SELECT *        -- which columns: all
FROM data       -- which table
WHERE ts >= now() - INTERVAL '3 months'
                -- which rows: the last 3 months
```

Filter in SQL, so only three months cross the network.

Read it back from disk

In [1]:

```
beehives = pd.read_parquet(RAW / "beehives.parquet")
sensors  = pd.read_parquet(RAW / "sensors.parquet")
raw_sensor_readings = pd.read_parquet(RAW / "data.parquet")
```

`data.parquet` becomes `raw_sensor_readings`: the name says it is raw and not yet cleaned. Everything from Exercise 1 on starts from these three dataframes.

Reading and writing files

Why Parquet, not CSV?

	CSV	Parquet
dtypes	**lost**	preserved
timestamps	become strings	stay timestamps
size	large	compressed
speed	slow	fast

Save `ts` to CSV and read it back and it is a **string**. Then `.dt.floor("h")` fails in Exercise 4 with an error message that never mentions CSV.

Exercise 1 · Profile

What is actually in these tables?

`head` · `tail` · `dtypes` · `value_counts` · `isna` · `describe` · `loc`

First look: head

In [2]:
```
raw_sensor_readings.head()
```

	reading_id	sensor_id	beehive_id	measurement_unit	ts	value
0	2236597	4	1	temperature	2026-07-28 10:28:39.165	30.7
1	2236598	4	1	relativeHumidity	2026-07-28 10:28:39.165	68.5
2	2236636	4	1	temperature	2026-07-28 10:28:39.165	30.7
3	2236637	4	1	relativeHumidity	2026-07-28 10:28:39.165	68.5
4	2236612	1	1	lightIntensity	2026-07-28 10:31:23.976	96248.0

In [3]:
```
raw_sensor_readings.head(10)
```

	reading_id	sensor_id	beehive_id	measurement_unit	ts	value
0	2236597	4	1	temperature	2026-07-28 10:28:39.165	30.7
1	2236598	4	1	relativeHumidity	2026-07-28 10:28:39.165	68.5
2	2236636	4	1	temperature	2026-07-28 10:28:39.165	30.7
3	2236637	4	1	relativeHumidity	2026-07-28 10:28:39.165	68.5
4	2236612	1	1	lightIntensity	2026-07-28 10:31:23.976	96248.0
5	2236613	1	1	rainGauge	2026-07-28 10:31:23.976	0.0
6	2236614	1	1	relativeHumidity	2026-07-28 10:31:23.976	33.0
7	2236615	1	1	temperature	2026-07-28 10:31:23.976	29.0
8	2236616	1	1	pressure	2026-07-28 10:31:23.976	100450.0
9	2236617	1	1	windDirection	2026-07-28 10:31:23.976	96.0

First look: tail and dtypes

In [4]:
```
raw_sensor_readings.tail()
```

	reading_id	sensor_id	beehive_id	measurement_unit	ts	value
515708	2752310	3	3	temperature	2026-10-28 10:18:10.743	12.5
515709	2752311	3	3	pressure	2026-10-28 10:18:10.743	100940.0
515710	2752312	3	3	windDirection	2026-10-28 10:18:10.743	354.0
515711	2752313	3	3	uvIndex	2026-10-28 10:18:10.743	2.9
515712	2752314	3	3	windSpeed	2026-10-28 10:18:10.743	1.1

In [5]:
```
raw_sensor_readings.dtypes
```

```
reading_id                    int64
sensor_id                     int64
beehive_id                    int64
measurement_unit                str
ts                  datetime64[us]
value                      float64
dtype: object
```

The index on the left of `tail()` is the row count: **515,713 rows**.

`ts` must be `datetime64`. Check this *first*, every time.

Selecting columns

```
raw_sensor_readings["value"]                  # one column -> Series
raw_sensor_readings[["ts", "value"]]          # several    -> DataFrame
```

One column: measurement_unit

In [6]:
```
raw_sensor_readings["measurement_unit"]
```

```
0              temperature
1         relativeHumidity
2              temperature
3         relativeHumidity
4           lightIntensity
                  ...
515708         temperature
515709            pressure
515710       windDirection
515711             uvIndex
515712           windSpeed
Name: measurement_unit, Length: 515713, dtype: str
```

In [7]:
```
raw_sensor_readings["measurement_unit"].value_counts()
```

```
measurement_unit
temperature          79343
relativeHumidity     79343
tempC3               39671
tempC1               39671
tempC2               39671
lightIntensity       39669
rainGauge            39669
pressure             39669
windDirection        39669
uvIndex              39669
windSpeed            39669
Name: count, dtype: int64
```

When, and is anything missing?

In [8]:
```
raw_sensor_readings["ts"].min()
```

```
Timestamp('2026-07-28 10:28:39.165000')
```

In [9]:
```
raw_sensor_readings["ts"].max()
```

```
Timestamp('2026-10-28 10:18:10.743000')
```

In [11]:
```
raw_sensor_readings["value"].isna().sum()
```

```
np.int64(0)
```

In [10]:
```
raw_sensor_readings["value"].isna()
```

```
0         False
1         False
2         False
3         False
4         False
          ...
515708    False
515709    False
515710    False
515711    False
515712    False
Name: value, Length: 515713, dtype: bool
```

Descriptive statistics: describe

In [12]:
```
raw_sensor_readings["value"].describe()
```

```
count    515713.000000
mean       8873.198813
std       27484.013765
min           0.000000
25%           2.200000
50%          26.300000
75%          62.700000
max      111404.000000
Name: value, dtype: float64
```

A `max` of 111,404 is light in lux, in the same column as °C and pascals. Group first.

In [13]:
```
raw_sensor_readings.groupby("measurement_unit")["value"].describe()
```

measurement_unit	count	mean	std	min	25%	50%	75%	max
lightIntensity	39669.0	14821.139530	24544.177910	0.0	0.0	992.0	18197.0	111404.0
pressure	39669.0	100106.224760	545.066852	98370.0	99700.0	100150.0	100470.0	101460.0
rainGauge	39669.0	0.128865	2.047008	0.0	0.0	0.0	0.0	152.4
relativeHumidity	79343.0	62.767007	15.955076	15.0	54.0	62.0	71.4	98.0
tempC1	39671.0	25.955280	6.226831	9.4	22.5	27.0	31.0	37.1
tempC2	39671.0	24.989229	5.669921	8.8	21.6	25.5	28.9	38.1
tempC3	39671.0	22.591253	7.023798	7.2	17.4	22.9	27.6	42.0
temperature	79343.0	23.696525	8.220606	3.9	16.8	24.9	31.3	43.9
uvIndex	39669.0	0.948453	1.840270	0.0	0.0	0.0	1.0	9.8
windDirection	39669.0	179.702110	115.009196	0.0	49.0	192.0	270.0	359.0
windSpeed	39669.0	0.542320	0.777330	0.0	0.0	0.0	1.0	7.0

The lookup tables

In [14]:
```
sensors.loc[0, "measurement_units"]
```

```
'lightIntensity,rainGauge,relativeHumidity,temperature,pressure,windDirection,uvIndex,windSpeed'
```

In [15]:
```
beehives.loc[0, "location"]
```

```
'(49.15182633037482,9.215298005933729)'
```

`.loc[row, column]` picks one value. `sensors` and `beehives` are tiny: look at them whole.

Four gotchas that will bite today

1 · NaN is not equal to itself

```
np.nan == np.nan      # False
df[df["x"] == np.nan]   # always empty
df[df["x"].isna()]      # correct
```

Most aggregations silently *skip* NaN: `mean()` of `[1, NaN, 3]` is `2.0`, not `NaN`.

2 · Chained assignment does nothing

```
df[df["x"] > 0]["y"] = 1         # df unchanged
df.loc[df["x"] > 0, "y"] = 1     # ✓
```

Since pandas 3, a filtered frame always behaves like a copy. Write through `.loc[rows, col]`, or build new frames with `.assign()`.

3 · Most methods do not mutate

```
df.drop_duplicates(subset=KEY)   # discarded!
df = df.drop_duplicates(subset=KEY)   # ✓
```

`rename`, `merge`, `sort_values`, `reset_index`, `fillna` — all return a **new** frame.

4 · Integers become floats

Introduce a single NaN into an `int64` column and the whole column becomes `float64`. IDs turn into `1.0`, `2.0` and stop matching on a join. Use the nullable `"Int64"` dtype if you need both.

Exercise 2 · Deduplicate

Name the grain. Let pandas check it.

`duplicated` · `drop_duplicates(subset=)`

Duplicates: what is one reading?

In [16]:
```
raw_sensor_readings.duplicated().sum()
```

```
0
```

In [17]:
```
raw_sensor_readings.head()
```

	reading_id	sensor_id	beehive_id	measurement_unit	ts	value
0	2236597	4	1	temperature	2026-07-28 10:28:39.165	30.7
1	2236598	4	1	relativeHumidity	2026-07-28 10:28:39.165	68.5
2	2236636	4	1	temperature	2026-07-28 10:28:39.165	30.7
3	2236637	4	1	relativeHumidity	2026-07-28 10:28:39.165	68.5
4	2236612	1	1	lightIntensity	2026-07-28 10:31:23.976	96248.0

Duplicates: drop_duplicates

In [18]:
```
KEYS = ["sensor_id", "measurement_unit", "ts", "value"]
```

In [19]:
```
before = len(raw_sensor_readings)
distinct = len(raw_sensor_readings.drop_duplicates(subset=KEYS))
print(f"{before:,} rows -> {distinct:,} distinct keys "
      f"({100 * (1 - distinct / before):.1f}% redundant)")
```

```
515,713 rows → 396,298 distinct keys (23.2% redundant)
```

In [20]:
```
distinct_sensor_readings = raw_sensor_readings.drop_duplicates(subset=KEYS)
```

Rows 0 and 2 are the same reading stored twice; only `reading_id` differs. So `duplicated()` on whole rows finds 0, and `subset=KEYS` is needed.

Exercise 3 · Reshape and join

Attach `role`. Split hive from weather. Collapse the weather.

`.map()` · `.assign()` · `merge(on=, how=)` · boolean filter · `drop_duplicates`

Why not just unstack?

A SHORTCUT, NOT IN THE NOTEBOOK

In [?]:

```
wide = (distinct_sensor_readings
        .set_index(["beehive_id", "ts", "measurement_unit"])["value"]
        .unstack("measurement_unit"))
wide[["temperature", "relativeHumidity", "windSpeed"]].head(4)
```

measurement_unit		temperature	relativeHumidity	windSpeed
beehive_id	ts			
1	2026-07-28 10:28:39.165	30.7	68.5	NaN
	2026-07-28 10:31:23.976	29.0	33.0	0.7
	2026-07-28 10:34:28.255	NaN	NaN	NaN
	2026-07-28 10:41:41.145	28.6	34.0	1.0

Two sensors, one column

30.7 is the brood sensor, 29.0 the weather station. Both are `temperature`. No error.

One device per row

Every device has its own `ts`: 72,243 rows, mostly `NaN`.

Weather three times

The station is registered to every hive, so each hive gets its own copy.

First step: hive or weather?

The hives and the weather station measure different places, so in the final table they become different columns. First split every reading: does it belong to a **hive** or to the **weather station**?

Way 1 · by `measurement_unit` ✕

Wind, UV, light, pressure and rain only come from the weather station. But `temperature` and `relativeHumidity` come from the weather station **and** the brood sensor, so the unit alone cannot tell them apart.

Way 2 · by device role ✓

Give every sensor a `role` from its `sensor_type`, attach it to each reading with a `merge`, then split on the role: `"weather"` or not.

Next: `.map` to assign the roles, `merge` to attach them, a boolean filter to split.

Mapping roles with `.map` and `.assign`

```
ROLE = {
    "LoRaWAN Dragino-S31-LB": "brood_sensor",
    "LoRaWAN Dragino-D23-LB": "probes",
    "LoRaWAN SenseCAP-S2120": "weather",
}
sensors = sensors.assign(
    role=sensors["sensor_type"].map(ROLE)
)
```

`.assign()` returns a **new** frame — it never mutates the original. Direct assignment (`sensors["role"] = ...`) also works.

SENSORS AFTER .MAP()

	sensor_id	beehive_id	sensor_type	role
0	1	1	LoRaWAN SenseCAP-S2120	weather
1	2	2	LoRaWAN SenseCAP-S2120	weather
2	3	3	LoRaWAN SenseCAP-S2120	weather
3	4	1	LoRaWAN Dragino-S31-LB	brood_sensor
4	5	3	LoRaWAN Dragino-S31-LB	brood_sensor
5	6	2	LoRaWAN Dragino-S31-LB	brood_sensor
6	7	3	LoRaWAN Dragino-D23-LB	probes
7	8	1	LoRaWAN Dragino-D23-LB	probes
8	9	2	LoRaWAN Dragino-D23-LB	probes

A key missing from the dict becomes `NaN` — so `sensors["role"].isna().sum()` is a free check that your mapping covered everything.

Look at the weather rows in `sensors` **before** you join. How many rows? How many distinct devices?

Join types: a small example

READINGS (LEFT)

	sensor_id	value
0	1	21.5
1	4	30.7
2	99	18.0

SENSORS (RIGHT)

	sensor_id	role
0	1	weather
1	4	brood_sensor
2	8	probes

Sensors 1 and 4 are in both tables. 99 is only in `readings`, 8 only in `sensors`.

HOW="INNER"

	sensor_id	value	role
0	1	21.5	weather
1	4	30.7	brood_sensor

Only keys in **both**. 99 and 8 are gone.

HOW="LEFT"

	sensor_id	value	role
0	1	21.5	weather
1	4	30.7	brood_sensor
2	99	18.0	NaN

Every **left** row. 99 has no role: `NaN`.

HOW="RIGHT"

	sensor_id	value	role
0	1	21.5	weather
1	4	30.7	brood_sensor
2	8	NaN	probes

Every **right** row. Sensor 8 has no reading: `NaN`.

HOW="OUTER"

	sensor_id	value	role
0	1	21.5	weather
1	4	30.7	brood_sensor
2	8	NaN	probes
3	99	18.0	NaN

Everything from both, gaps filled with `NaN`.

merge: joining two tables

In [21]:

```
SENSOR_COLUMNS = ["sensor_id", "beehive_id", "role"]
ON_COLUMNS = ["sensor_id", "beehive_id"]
MERGE_TYPE = "left"

before = len(distinct_sensor_readings)
distinct_sensor_readings = distinct_sensor_readings.merge(
    sensors[SENSOR_COLUMNS],
    on=ON_COLUMNS,
    how=MERGE_TYPE
)
print(f"distinct_sensor_readings {before:,} -> {len(distinct_sensor_readings):,} after merge")
```

```
distinct_sensor_readings 396,298 → 396,298 after merge
```

`how="left"` keeps every left row · `"inner"` only matches · `"outer"` everything · `"right"` every right row

A careless join

In [22]:
```
len(distinct_sensor_readings.merge(sensors[["beehive_id", "role"]], on="beehive_id", how="left"))
```

```
1188894
```

Joined on the hive, not the device: each hive has 3 sensors, so every row is **tripled**. No error, no warning.

Let pandas check for you: `merge(..., validate="many_to_one")` raises if the right side is not unique on the key.

Filtering rows: hive vs weather

In [23]:
```
mask = sensors['role'] == 'weather'
sensors[mask]
```

	sensor_id	beehive_id	sensor_name	sensor_type	measurement_units	installation_date	last_seen	is_active	role
0	1	1	LoRa-2CF7F1C0613005BC	LoRaWAN SenseCAP-S...	lightIntensity,rai...	2025-04-14	2025-06-13 13:51:12...	True	weather
1	2	2	LoRa-2CF7F1C0613005BC	LoRaWAN SenseCAP-S...	lightIntensity,rai...	2025-04-14	2025-06-13 13:51:12...	True	weather
2	3	3	LoRa-2CF7F1C0613005BC	LoRaWAN SenseCAP-S...	lightIntensity,rai...	2025-04-14	2025-06-13 13:51:12...	True	weather

In [24]:
```
hive_rows = distinct_sensor_readings[distinct_sensor_readings["role"] != "weather"]
weather_rows = distinct_sensor_readings[distinct_sensor_readings["role"] == "weather"]
```

One weather station, stored three times

In [25]:
```
weather_rows["sensor_id"].unique()
```

```
array([1, 2, 3])
```

In [26]:
```
weather_rows.sort_values("measurement_unit").head(6)
```

	reading_id	sensor_id	beehive_id	measurement_unit	ts	value	role
2	2236612	1	1	lightIntensity	2026-07-28 10:31:23.976	96248.0	weather
10	2236620	2	2	lightIntensity	2026-07-28 10:31:23.976	96248.0	weather
18	2236628	3	3	lightIntensity	2026-07-28 10:31:23.976	96248.0	weather
34	2236651	1	1	lightIntensity	2026-07-28 10:41:41.145	97400.0	weather
42	2236659	2	2	lightIntensity	2026-07-28 10:41:41.145	97400.0	weather
50	2236667	3	3	lightIntensity	2026-07-28 10:41:41.145	97400.0	weather

In [27]:
```
# How many weather rows share the same (measurement_unit, ts)?
weather_rows = weather_rows.drop_duplicates(subset=["measurement_unit", "ts"])
```

In [28]:
```
weather_rows = weather_rows.drop(columns=["sensor_id", "beehive_id"])
```

THE ONE HABIT THAT MATTERS

Check your row count after every join

If it changes and you **cannot say exactly why** — stop and find out.

```
print(len(df))            # before
df = df.merge(other, on="sensor_id", how="left")
print(len(df))            # after — did that number move? should it have?
```

That single reflex catches the most expensive mistake in this dataset.

Exercise 4 · Align time and handle gaps

One clock. Long to wide. Every hive-hour, even the silent ones.

`.dt.floor("h")` · `groupby().mean()` · `unstack` · `reindex` · `merge`

Floor timestamps to the hour

In [29]:
```
hive_rows = hive_rows.assign(hour=hive_rows["ts"].dt.floor("h"))
hive_rows[["ts", "hour"]].head()
```

	ts	hour
0	2026-07-28 10:28:39.165	2026-07-28 10:00:00
1	2026-07-28 10:28:39.165	2026-07-28 10:00:00
26	2026-07-28 10:34:28.255	2026-07-28 10:00:00
27	2026-07-28 10:34:28.255	2026-07-28 10:00:00
28	2026-07-28 10:34:28.255	2026-07-28 10:00:00

> `.dt` unlocks datetime methods on a whole column. Do the same for `weather_rows`.

groupby + mean: still long

In [30]:
```
hive_hourly = hive_rows.groupby(["beehive_id", "hour", "measurement_unit"])["value"].mean()
hive_hourly.head()
```

```
beehive_id  hour                 measurement_unit
1           2026-07-28 10:00:00  relativeHumidity    65.70
                                 tempC1              32.70
                                 tempC2              29.05
                                 tempC3              30.20
                                 temperature         30.85
Name: value, dtype: float64
```

One value per hive, hour and measurement. Look at the **index**: three levels, still one measurement per row.

Long → wide: unstack

In [31]:
```
hive_hourly = hive_hourly.unstack("measurement_unit")
hive_hourly.head()
```

measurement_unit		relativeHumidity	tempC1	tempC2	tempC3	temperature
beehive_id	hour					
1	2026-07-28 10:00:00	65.700000	32.700000	29.050000	30.2	30.850000
	2026-07-28 11:00:00	61.266667	33.466667	30.133333	31.5	31.433333
	2026-07-28 12:00:00	59.566667	34.033333	31.433333	32.9	31.766667
	2026-07-28 13:00:00	59.600000	35.066667	32.666667	33.9	32.433333
	2026-07-28 14:00:00	57.500000	35.150000	33.450000	34.6	33.233333

SAME THREE VERBS FOR WEATHER

In [32]:
```
weather_rows = weather_rows.assign(hour=weather_rows["ts"].dt.floor("h"))
weather_hourly = weather_rows.groupby(["hour", "measurement_unit"])["value"].mean()
```

In [33]:
```
weather_hourly = weather_hourly.unstack("measurement_unit").add_prefix("outside_")
```

Weather has no hive: its grain is just `hour`. `add_prefix` stops `temperature` clashing with the brood sensor's.

Building a complete index: date_range + reindex

Invisible NULLs. A power outage or a broken sensor leaves no NULL behind: the row is simply missing. A complete grid turns every missing hour into a row of `NaN` you can count.

In [34]:
```
START_TIME_H = distinct_sensor_readings["ts"].min().floor("h")
END_TIME_H = distinct_sensor_readings["ts"].max().floor("h")
hours = pd.date_range(START_TIME_H, END_TIME_H, freq="h", name="hour")
```

EVERY HIVE × EVERY HOUR

In [35]:
```
grid = pd.MultiIndex.from_product(
    [beehives["beehive_id"], hours], names=["beehive_id", "hour"]
)
features = hive_hourly.reindex(grid)
print(f"hive hours: {len(hive_hourly):,} observed vs {len(features):,} complete")
```

```
hive hours: 6,564 observed vs 6,627 complete
```

WEATHER: EVERY HOUR, NO HIVE

In [36]:
```
observed = len(weather_hourly)
weather_hourly = weather_hourly.reindex(hours)
print(f"weather hours: {observed:,} observed vs {len(weather_hourly):,} complete")
```

```
weather hours: 2,209 observed vs 2,209 complete
```

Measuring gaps

In [37]:
```
hive_hourly.reset_index().groupby("beehive_id")["hour"].diff()
```

```
0                  NaT
1      0 days 01:00:00
2      0 days 01:00:00
3      0 days 01:00:00
4      0 days 01:00:00
              ...
6559   0 days 01:00:00
6560   0 days 01:00:00
6561   0 days 01:00:00
6562   0 days 01:00:00
6563   0 days 01:00:00
Name: hour, Length: 6564, dtype: timedelta64[us]
```

THE LONGEST GAP

In [38]:
```
hive_hourly.reset_index().groupby("beehive_id")["hour"].diff().max()
```

```
Timedelta('0 days 05:00:00')
```

> The time since the previous observed hour, per hive. `1 hour` means no gap. Missing hours stay `NaN`.

Attach the weather: merge on hour

In [39]:

```
before = len(features)
features = features.reset_index().merge(
    weather_hourly.reset_index(), on="hour", how="left", validate="many_to_one"
)
print(f"features {before:,} -> {len(features):,} after weather merge")
```

```
features 6,627 → 6,627 after weather merge
```

> One weather hour attaches to all three hives. `validate="many_to_one"` proves weather has one row per hour, so the row count cannot grow.

Exercise 5 · Validate and document

Local time. Target names. Prove the grain, then save.

`tz_localize` / `tz_convert` · `rename` · `assert` · `to_parquet`

Time zones: localize, then convert

In [40]:
```
local = features["hour"].dt.tz_localize("UTC").dt.tz_convert("Europe/Berlin")
```

DID THIS WINDOW CROSS A CLOCK CHANGE?

In [41]:
```
local.dt.strftime("%z").unique()
```

```
<ArrowStringArray>
['+0200', '+0100']
Length: 2, dtype: str
```

In [42]:
```
features["hour_local"] = local.dt.hour
features["month"] = local.dt.month
features["dayofweek"] = local.dt.dayofweek
```

`tz_localize` **attaches** a zone. `tz_convert` **moves** the clock. Never hardcode +01/+02.

Rename to the target schema

In [43]:

```
features = features.rename(columns={
    "tempC1": "brood_probe_temp", "tempC2": "food_probe_temp", "tempC3": "top_probe_temp",
    "temperature": "brood_temp", "relativeHumidity": "brood_humidity",
    "outside_temperature": "outside_temp",
    "outside_relativeHumidity": "outside_humidity",
    "outside_windSpeed": "outside_wind_speed",
    "outside_windDirection": "outside_wind_dir",
    "outside_uvIndex": "outside_uv_index",
    "outside_pressure": "outside_pressure_pa",
    "outside_lightIntensity": "outside_light",
    "outside_rainGauge": "outside_rain",
})
```

> The validator expects **exactly** these names. A typo in a key is silently ignored: check `features.columns`.

Save and validate

In [44]:

```python
assert not features.duplicated(subset=["beehive_id", "hour"]).any()
assert len(features) == len(grid)
features.to_parquet(OUT / "features_hourly.parquet", index=False)
```

THEN, IN A TERMINAL FROM THE REPO ROOT

```
uv run python workshop/validate_features.py workshop/out/features_hourly.parquet
```

```
Checking workshop/out/features_hourly.parquet  (6,627 rows x 18 columns)

  [PASS] columns present              all 18 present
  [PASS] one row per hive-hour        no duplicates
  [PASS] row count                    6,627 = 3 hives x 2,209 hours
  [PASS] hourly grid complete         2,209 consecutive hours
  [PASS] values in range              every column inside plausible bounds
  [PASS] missingness sane             worst column brood_temp at 6.7% -- gaps kept honest
  [PASS] weather shared across hives  one ambient value per hour

  All checks passed.
```

Homework: write `DATA_DICTIONARY.md` . Every column and its unit, and every decision: the window you extracted, what you dropped, what you left empty, and why.

Exercise 6 · Look at what you built

Check the table with your eyes. Answer the materials question.

`plt.plot` · `plt.subplots` · `df.plot` · `ax.axhline` · `sns.lineplot(hue=)` · `sns.set_palette`

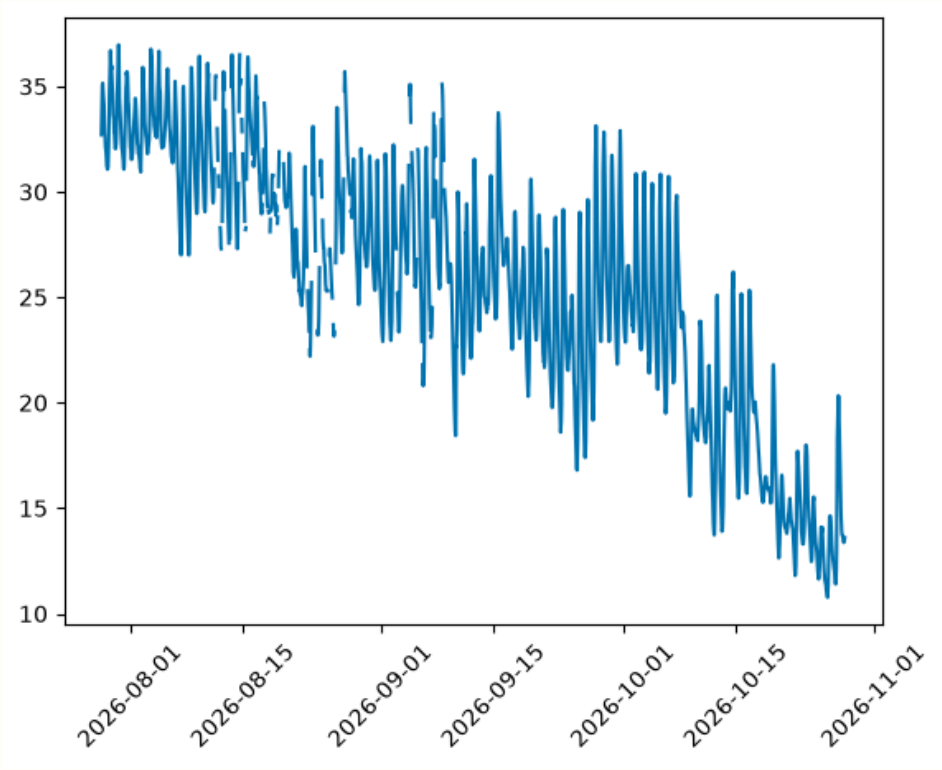
Step 1 · Your first plot: `plt.plot(x, y)`

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_palette("colorblind")
OPTIMAL_C = 35
DAYS = 7
HIVE = 1

one = features[
    features["beehive_id"] == HIVE]
plt.plot(one["hour"], one["brood_probe_temp"])
plt.xticks(rotation=45)  # tilt the dates
plt.show()
```

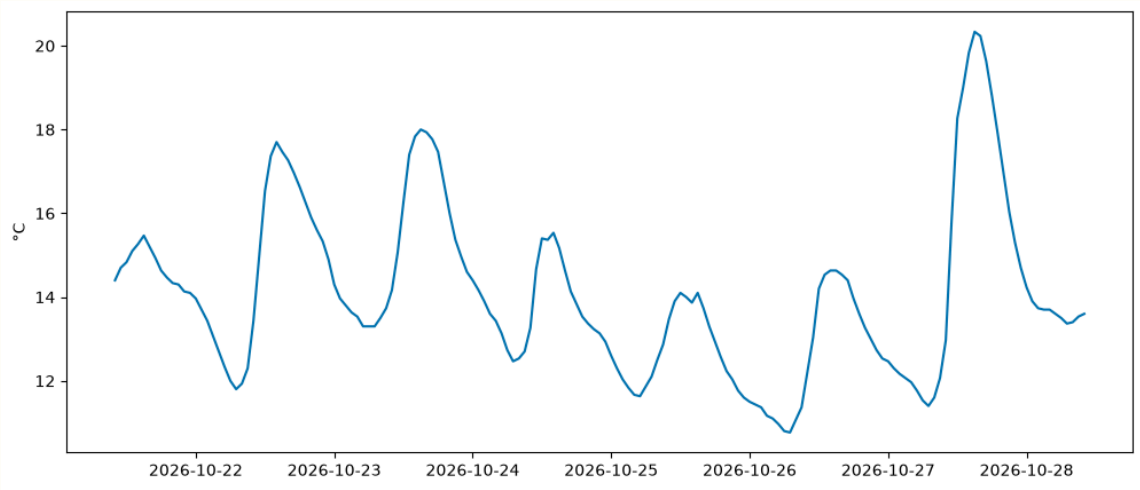
`plt.plot` takes an **x** and a **y**, both columns of the same length. One hive, one probe, the whole extract.



Three months of hourly points is a **scribble**. You see the trend, but not a single day.

Step 2 · Zoom in: a time window and `fig, ax`

```
window = one[one["hour"] >= one["hour"].max() - pd.Timedelta(days=DAYS)]
fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(window["hour"], window["brood_probe_temp"])
ax.set_ylabel("°C")
```



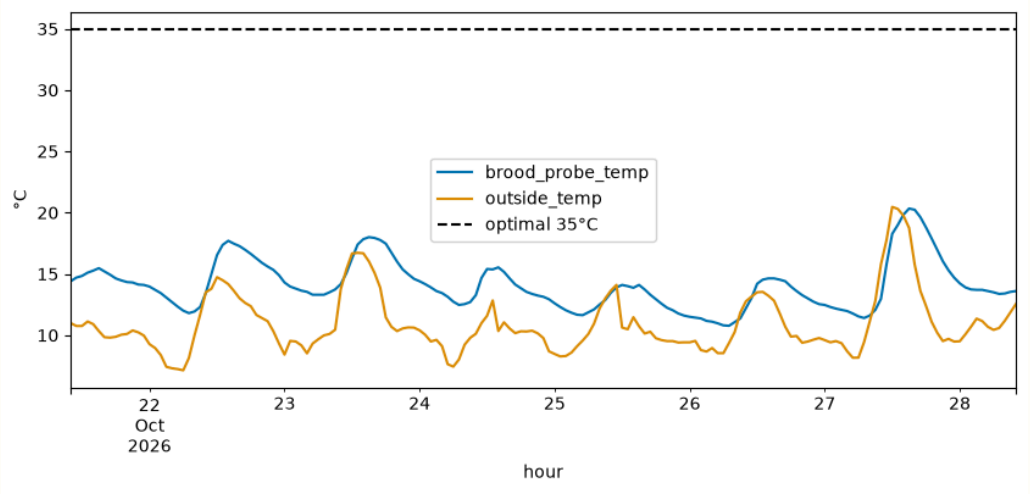
`plt.subplots()` returns the **figure** (the whole picture) and an **axes** (one plot inside it). From now on you draw on `ax`.

The window is the same boolean filter as Exercise 3, using a time: "everything after the last hour minus 7 days". Now each day is visible.

Step 3 · Let pandas draw, then add a reference line

```
ax = window.plot(
    x="hour",
    y=["brood_probe_temp", "outside_temp"],
    figsize=(10, 4))
ax.axhline(OPTIMAL_C, color="black",
           linestyle="--",
           label=f"optimal {OPTIMAL_C}°C")
ax.set_ylabel("°C")
ax.legend()
```

Same hive, same window: the only new thing is the code.



`df.plot` is matplotlib underneath: two columns become two lines, with a legend and formatted dates. It **returns the axes**, so `axhline` can draw the target on it. Make the line **black**: left at the default it takes the first palette colour, the same blue as brood.

How to read the chart

What to look for

- Brood should hug 35 °C more tightly than outside does.
- Breaks in the line are leftover gaps, not a plotting bug. Matplotlib skips `NaN`.
- Try `DAYS = 7`, then `3`. Comment out the window line to see the whole extract again.

If it looks wrong

If the two traces sit on top of each other, you plotted the same column twice or joined weather onto brood by mistake.

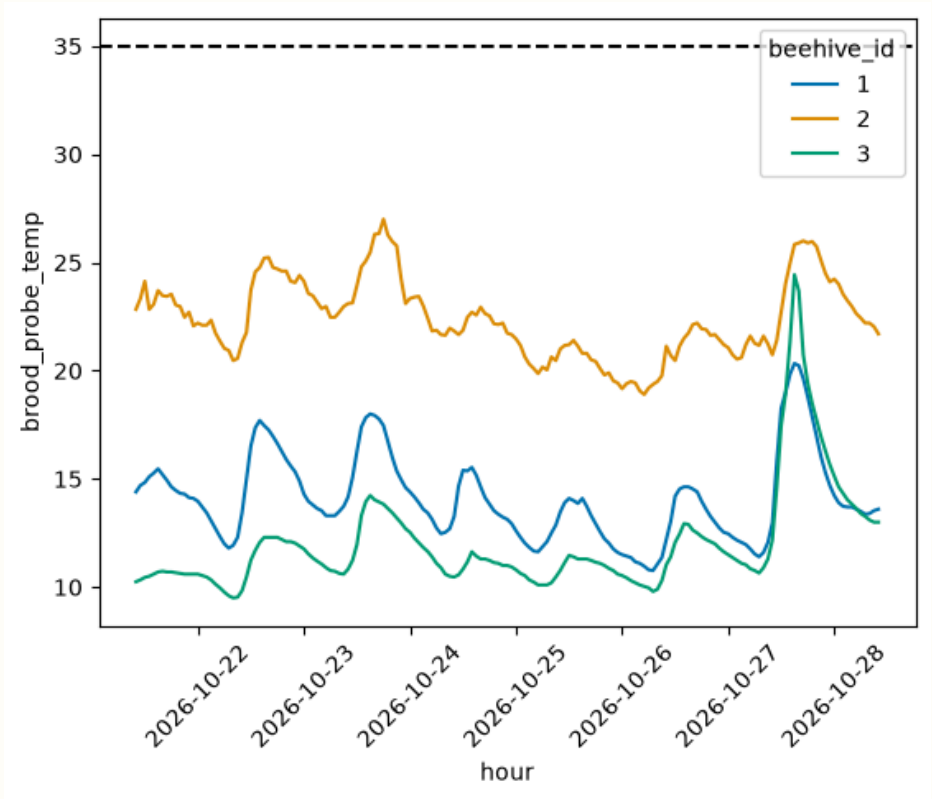
If there is no dashed line, `axhline` was never called. The 35 °C line is the point of the chart. Without it you cannot tell whether brood is doing its job.

Step 4 · seaborn: one line per hive with `hue`

```
temp = features[[
    "beehive_id", "hour", "brood_probe_temp",
    "food_probe_temp", "top_probe_temp",
    "outside_temp"]]
temp_7_days = temp[temp["hour"] >=
                   temp["hour"].max()
                   - pd.Timedelta(days=DAYS)]

ax = sns.lineplot(data=temp_7_days,
                  x="hour",
                  y="brood_probe_temp",
                  hue="beehive_id",
                  palette="colorblind")
ax.axhline(OPTIMAL_C, color="black",
           linestyle="--")
plt.xticks(rotation=45)
```

seaborn takes the **long** frame, and `hue=` splits it into one coloured line per hive. No filtering per hive.

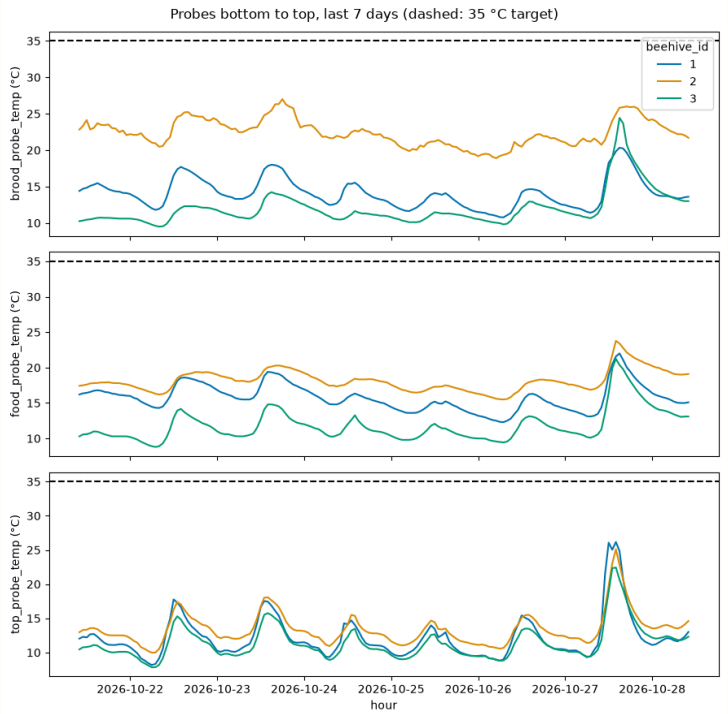


Step 5 · One panel per level: `plt.subplots`

```
PROBES = ["brood_probe_temp", "food_probe_temp",
          "top_probe_temp"]   # bottom to top
fig, axes = plt.subplots(3, 1, figsize=(9, 9),
                         sharex=True)

# axes: three plots. enumerate: 0, 1, 2
for i, ax in enumerate(axes):
    sns.lineplot(
        data=temp_7_days, x="hour",
        y=PROBES[i],
        hue="beehive_id", palette="colorblind",
        ax=ax,
        legend=(i == 0),  # top panel only
    )
    ax.axhline(OPTIMAL_C, color="black",
               linestyle="--")
    ax.set_ylabel(f"{PROBES[i]} (°C)")

fig.suptitle(f"Probes bottom to top, last {DAYS} days "
             f"(dashed: {OPTIMAL_C} °C target)")
fig.tight_layout()
```



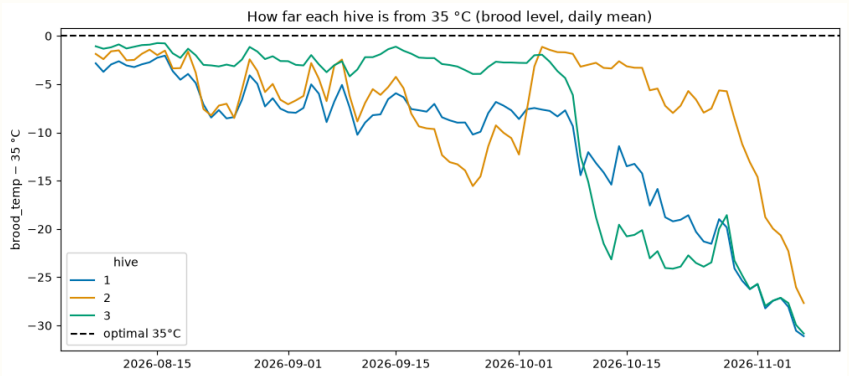
`ax=ax` picks the panel. `legend=(i == 0)` keeps one legend instead of three.

Step 6 · How far from 35 °C?

```
deviation = features.assign(
    deviation=features["brood_temp"] - OPTIMAL_C,
    day=features["hour"].dt.floor("D"),
)
daily_dev = (deviation
             .groupby(["beehive_id", "day"])["deviation"]
             .mean().reset_index())

fig, ax = plt.subplots(figsize=(12, 5))
sns.lineplot(data=daily_dev, x="day", y="deviation",
             hue="beehive_id", palette="colorblind",
             ax=ax)
ax.axhline(0, color="black", linestyle="--")
```

`brood_temp` is the sensor in the brood level. Subtract 35: **0** is perfect, below 0 is too cold.



Every line stays below 0: no hive reaches 35 °C. Which one stays **closest** changes over the season.

Step 7 · One number per hive

```
mae = (deviation
       .assign(abs_dev=deviation["deviation"].abs())
       .groupby("beehive_id")["abs_dev"].mean()
       .round(2)
       .sort_values())
mae
```

`.abs()` makes every distance positive, so too cold and too hot both count. The mean of that is the **mean absolute deviation**: lower is better.

OUTPUT

```
beehive_id
2     7.37
3     9.29
1    11.43
Name: abs_dev, dtype: float64
```

Hive 2 (thermofolie) is closest **on average**. The Step 6 chart shows it wins only from October on. One number hides *when*.

Answering the materials question

Read the Step 5 – 7 charts

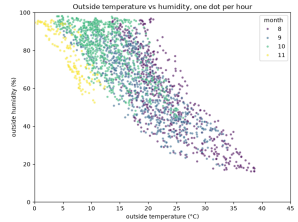
1. The top level is empty. Does it follow the brood or the outside?
2. Did one hive's sensors stop sending data for a while, while the other hives kept reporting? Hint: the daily means hide gaps. Plot the full extract hourly, or count the missing hours per hive with `.isna().sum()`.
3. The data covers August to November. Would the same hive still be best in the cold of winter? What data would you need to be sure?

> A chart shows a pattern, not a verdict. One week of late October is one week. Say what the data supports *and* what it cannot tell you yet.

Done? Tell us how it went.
Scan to leave feedback on the workshop.

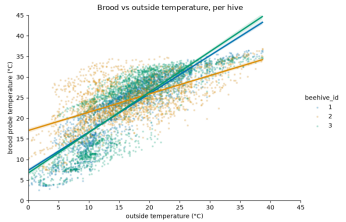
Optional · six more kinds of plot

Done early? The end of the notebook has one cell per plot type, already working. Run them and read each chart.



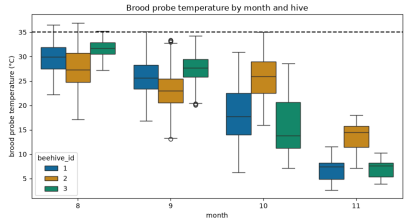
Scatter · `sns.scatterplot`

Two columns against each other: hot hours are dry.



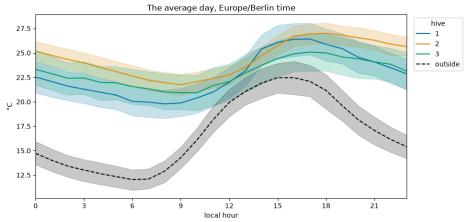
Scatter + line · `sns.lmplot`

Flatter line per hive = better insulated.



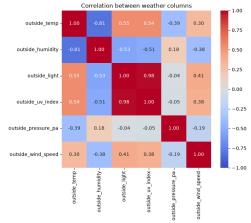
Boxplot · `sns.boxplot`

Median and spread per group: the brood cools month by month.



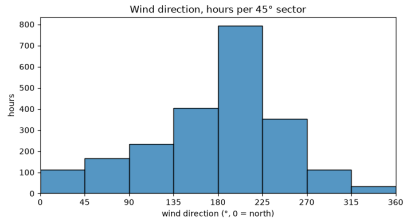
Line with a band · `sns.lineplot`

The average day: the mean plus its uncertainty.



Heatmap · `sns.heatmap`

Which columns move together: light and UV almost the same.



Histogram · `sns.histplot`

How often each value occurs: wind mostly from the south-west.